Introduction to Computer Architecture and Systems

Chapter 7
Introduction to High-Level Language

Fengkai Wang(Iori Oikawa), Fall 2025 @ Drew University

Previously of CSCI 260…

- Full Stack and Queue Protocol
- Stack in Java: Java Virtual Machine (JVM)
- Sequential Processors (CPU) and Parallel Processors (GPU)

Assignment 2

Implement Array and Linked List

Deadline: 11/17 23:50

- Look up for the definition of Array and Linked List, in your comfortable language. Summarize your understanding by creating an one-page reference.

- Based on the reference, translate the definition and behavior of the array and the linked list into series of ISA instructions. Correctly defining the functional component is important. Make good organization by using labels wisely.

- Deliverables:
  1 - Reference of Array and Linked List, one page each, which contains: Definition, Features, Operations/Behaviors. (Graphics are encouraged)
  2 - LC-3 Code Snippets, with corresponding documentation to introduce how to use them.

Course Project: Assembly

Project Preview: Calculator

Note: You could pick either Assembly project or C project.

- Create a 4 digit calculator with LC-3.
- The calculator must perform plus, minus, multiply, and divide.
- You can refer to the Chapter 10 of the textbook.
- You have to organize the control flow by yourself.
- You might need the stack and queue protocols.
- [Deadline] 11/30, 23:59.
- Deliverable:
  Video Demo (under 2 minutes)
  Code (.asm, .obj, and .sym)

High Level Language: What is it?

high-level language =
human readable language

low-level language =
machine-oriented language

However, determining high level or low level language is relative.

So… is C a low level language?

It depends on context, actually.

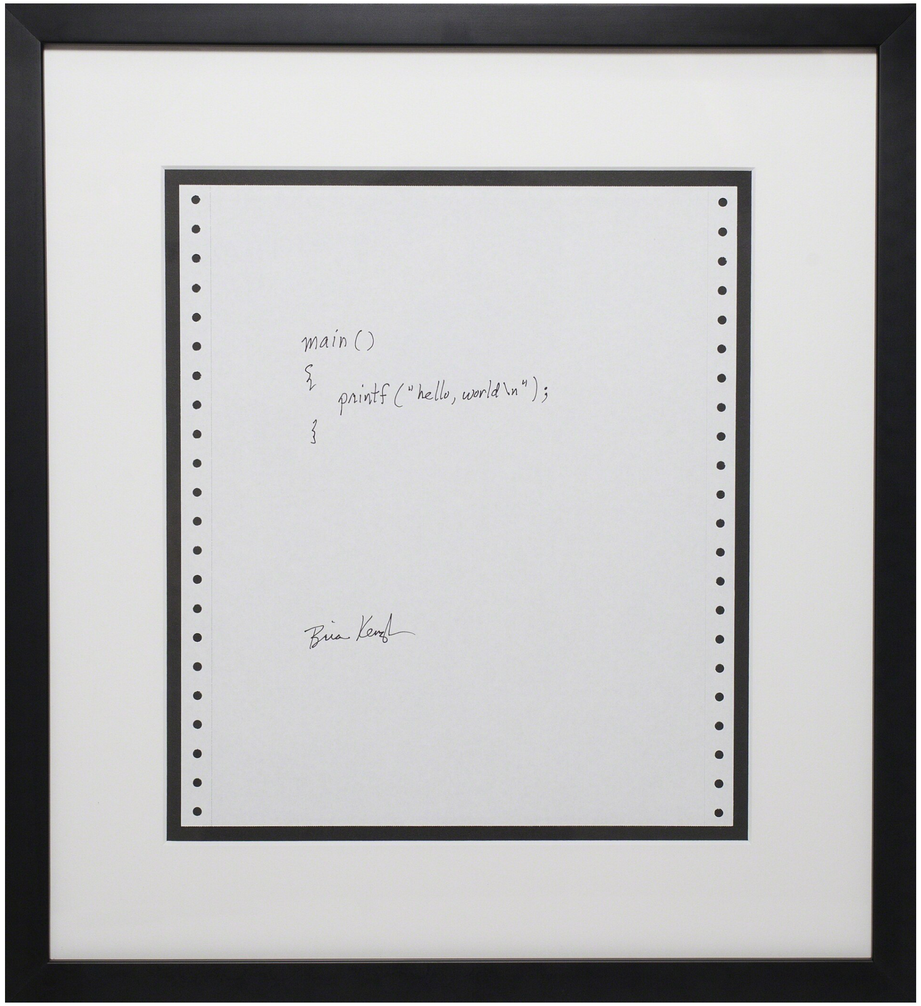
- Compared to PHP, C is low level;
- Compared to x86 assembly, C is high level;
- Compared to the instructions I used to construct this universe in which you reside, C is so high level you can barely see it.

Takeaway: C is much human readable language.

C Programming Language

History of C

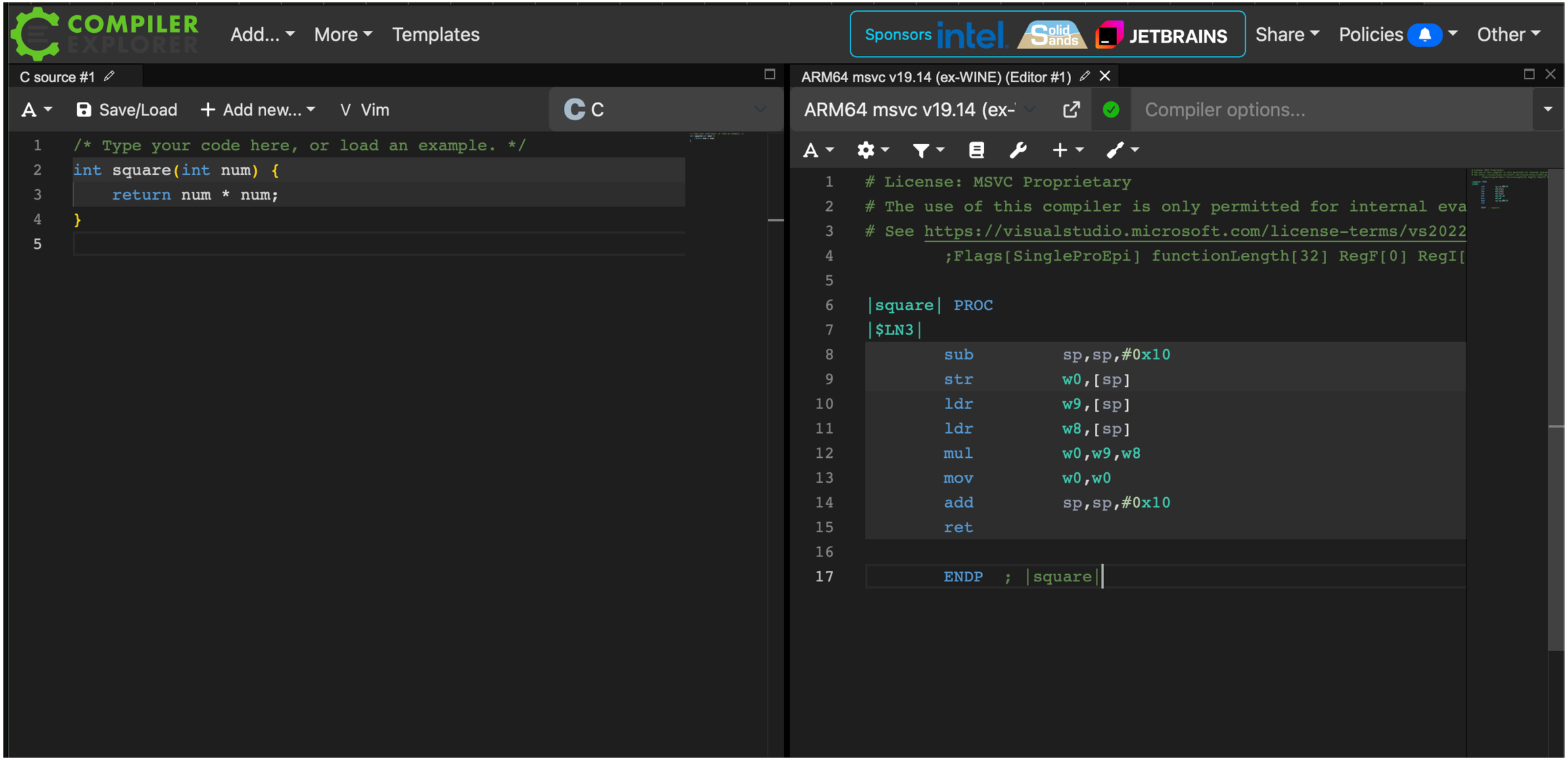
- 1972, at Bell Labs (Summit, NJ).
- Motivation: Re-implementing the kernel of the Unix operating system.
- Influenced by: B (BCPL, CPL), ALGOL 68, PL/I, Fortran
- Influenced: AMPL, AWK, csh, C++, C--, C#, Objective-C, D, Go, Java, JavaScript, JS++, Julia, Limbo, LPC, Perl, PHP, Pike, Processing, Python, Rust, V (Vlang), Vala, Verilog (HDL), Nim, Zig…



Why should we learn C?

You should learn C because it's a foundational language that provides a deep understanding of how computers work, leading to more efficient and effective programming in other languages. Learning C offers advantages such as direct hardware access, high performance, and the ability to work in fields like embedded systems and system programming, making you a more versatile and skilled developer.

Your new toy:
godbolt.org, online compiler


COMPILER EXPLORER
Add...
More
Templates
Sponsors intel Solid Sands JETBRAINS
Share
Policies
Other
C source #1
A
Save/Load
Add new...
V Vim
C
1 /* Type your code here, or load an example. */
2 int square(int num) {
3 return num * num;
4 }
5
ARM64 msvc v19.14 (ex-WINE) (Editor #1)
ARM64 msvc v19.14 (ex-
Compiler options...
1 # License: MSVC Proprietary
2 # The use of this compiler is only permitted for internal eva
3 # See https://visualstudio.microsoft.com/license-terms/vs2022
4 ;Flags[SingleProEpi] functionLength[32] RegF[0] RegI[
5
6 |square| PROC
7 |$LN3|
8 sub sp,sp,#0x10
9 str w0,[sp]
10 ldr w9,[sp]
11 ldr w8,[sp]
12 mul w0,w9,w8
13 mov w0,w0
14 add sp,sp,#0x10
15 ret
16
17 ENDP ; |square|

Goal of C, revisited…

Goal of C, Revisited

… in comparison of Java

Java Design Goals (1995)	C Design Goals (1975)
Language of the Internet	Compose Unix OS
High-level; insulated from hardware and OS	Low-level; close to HW and OS
Good for application-level programming	Good for system-level programming
Support object-oriented programming	Support structured programming
Safe: can't step "outside the sandbox"	Unsafe: don't get in the programmer's way
Look like C!	

From Java to C

From Java to C: Efficiency

- Java has automatic array-bounds checking, nullpointer checking, automatic memory management (garbage collection), other safety features;
- C has manual bounds checking, null checking, memory management.

C programs are (often) faster, but sometimes buggy, exploitable

From Java to C: Conclusion

	Java	C
Portability	+	-
Efficiency	~	+
Safety	+	-

From Java to C: Details

	Java	C
Overall Program Structure	Hello.java: public class Hello { public static void main (String[] args) { System.out.println("hello, world"); } }	hello.c: #include <stdio.h> int main(void) { printf("hello, world\n"); return 0; }
Building	$ javac Hello.java	$ gcc217 hello.c -o hello
Running	$ java Hello hello, world $	$ ./hello hello, world $

From Java to C: Details

	Java	C
Character type	`char // 16-bit Unicode`	`char /* 8 bits */`

Integral types	`byte // 8 bits`
	`short // 16 bits`
	`int // 32 bits`
	`long // 64 bits`
		`(unsigned) char`
		`(unsigned) short`
		`(unsigned) int`
		`(unsigned) long`

Floating point types	`float // 32 bits`
	`double // 64 bits`
		`float`
		`double`
		`long double`

Logical type	`boolean`	`/* no equivalent */` `/* use integral type */`
Generic pointer type	`Object`	`void*`

Constants	`final int MAX = 1000;`	`#define MAX 1000`
		`const int MAX = 1000;`
		`enum {MAX = 1000};`

From Java to C: Details

	Java	C

Arrays	`int [] a = new int [10];`
	`float [][] b =`
	`  new float [5][20];`
		`int a[10];`
		`float b[5][20];`

Array bound checking	`// run-time check`	`/* no run-time check */`
Pointer type	`// Object reference is an` `// implicit pointer`	`int *p;`

Record type	`class Mine`
	`{  int x;`
	`  float y;`
	`}`
		`struct Mine`
		`{  int x;`
		`  float y;`
		`};`

From Java to C: Details

	Java	C

Strings	`String s1 = "Hello";`
	`String s2 = new`
	`  String("hello");`
		`char *s1 = "Hello";`
		`char s2[6];`
		`strcpy(s2, "hello");`

String concatenation	`s1 + s2` `s1 += s2`	`#include <string.h>` `strcat(s1, s2);`
Logical ops *	`&&, ||, !`	`&&, ||, !`
Relational ops *	`=, !=, >, <, >=, <=`	`=, !=, >, <, >=, <=`
Arithmetic ops *	`+, -, *, /, %, unary -`	`+, -, *, /, %, unary -`
Bitwise ops	`>>, <<, >>>, &, |, ^`	`>>, <<, &, |, ^`

Assignment ops	`=, *=, /=, +=, -=, <<=,`
	`>>=, >>>=, =, &=, ^=, |=,`
	`%=`
		`=, *=, /=, +=, -=, <<=,`
		`>>=, =, &=, ^=, |=, %=`

From Java to C: Details

	Java	C

if stmt *	`if (i < 0)`
	`  *statement1;*`
	`else`
	`  *statement2;*`
		`if (i < 0)`
		`  *statement1;*`
		`else`
		`  *statement2;*`

switch stmt *	`switch (i)`
	`{  case 1:`
	`  ...`
	`  break;`
	`  case 2:`
	`  ...`
	`  break;`
	`  default:`
	`  ...`
	`}`
		`switch (i)`
		`{  case 1:`
		`  ...`
		`  break;`
		`  case 2:`
		`  ...`
		`  break;`
		`  default:`
		`  ...`
		`}`

goto stmt	`// no equivalent`	`goto someLabel;`

From Java to C: Details

	Java	C

for stmt	`for (int i=0; i<10; i++)`
	`  statement;`
		`int i;`
		`for (i=0; i<10; i++)`
		`  statement;`

while stmt *	`while (i < 0)` `  statement;`	`while (i < 0)` `  statement;`

do-while stmt *	`do`
	`  statement;`
	`while (i < 0)`
		`do`
		`  statement;`
		`while (i < 0);`

continue stmt *	`continue;`	`continue;`
labeled continue stmt	`continue someLabel;`	`/* no equivalent */`
break stmt *	`break;`	`break;`
labeled break stmt	`break someLabel;`	`/* no equivalent */`

From Java to C: Details

	Java	C
return stmt *	`return 5;` `return;`	`return 5;` `return;`

Compound stmt (alias block) *	`{`
	`  statement1;`
	`  statement2;`
	`}`
		`{`
		`  statement1;`
		`  statement2;`
		`}`

Exceptions	`throw, try-catch-finally`	`/* no equivalent */`
Comments	`/* comment */` `// another kind`	`/* comment */`

Method / function call	`f(x, y, z);`
	`someObject.f(x, y, z);`
	`SomeClass.f(x, y, z);`
		`f(x, y, z);`

Sample Program in C

```
#include <stdio.h>
#include <stdlib.h>

int main(void)
{  const double KMETERS_PER_MILE = 1.609;
   int miles;
   double kMeters;

   printf("miles: ");
   if (scanf("%d", &miles) != 1)
   {  fprintf(stderr, "Error: Expected a number.\n");
      exit(EXIT_FAILURE);
   }

   kMeters = (double)miles * KMETERS_PER_MILE;
   printf("%d miles is %f kilometers.\n",
      miles, kMeters);
   return 0;
}
```

Elements of C

Sample Program in C

```
#include <stdio.h>
#include <stdlib.h>

int main(void)
{  const double KMETERS_PER_MILE = 1.609;
   int miles;
   double kMeters;

   printf("miles: ");
   if (scanf("%d", &miles) != 1)
   {  fprintf(stderr, "Error: Expected a number.\n");
      exit(EXIT_FAILURE);
   }

   kMeters = (double)miles * KMETERS_PER_MILE;
   printf("%d miles is %f kilometers.\n",
      miles, kMeters);
   return 0;
}
```

Another Program in C: charcount

```
#include <stdio.h>
/* Write to stdout the number of
   chars in stdin. Return 0. */
int main(void)
{   int c;
    int charCount = 0;
    c = getchar();
    while (c != EOF)
    {   charCount++;
        c = getchar();
    }
    printf("%d\n", charCount);
    return 0;
}
```

‘Build’ C Programs

charcount.c

```
#include <stdio.h>
/* Write to stdout the number of
   chars in stdin. Return 0. */
int main(void)
{  int c;
   int charCount = 0;
   c = getchar();
   while (c != EOF)
   {  charCount++;
      c = getchar();
   }
   printf("%d\n", charCount);
   return 0;
}
```

Build charcount.c

Use gcc as compiler by calling it… To make your source code, charcount.c…



```
gcc charcount.c -o charcount
```

…into an executable file contains machine code as the output… ‘charcount’

Finally, run your code by typing

```
./charcount
```

Run charcount

```
$ ./charcount
  Line 1
  Line 2
  ^D
  14
$
```

Run charcount… fancier ways

```
$ cat somefile
Line 1
Line 2
$ ./charcount < somefile
14
$
```

Run charcount… other fancier ways

```
$ ./charcount > someotherfile
Line 1
Line 2
^D
$ cat someotherfile
14
```

Compilation Process in C

What happens when you issue the gcc command?

- The compiler is doing 4 step compilation process:
- 1. Preprocess
- 2. Compile
- 3. Assemble
- 4. Link

The starting point

```
#include <stdio.h>
/* Write to stdout the number of
   chars in stdin. Return 0. */
int main(void)
{  int c;
   int charCount = 0;
   c = getchar();
   while (c != EOF)
   {  charCount++;
      c = getchar();
   }
   printf("%d\n", charCount);
   return 0;
}
```

- C language
- No definitions of getchar() and printf()

Step 1 - Preprocessing

`gcc -E charcount.c > charcount.i`

```
#include <stdio.h>
/* Write to stdout the number of
   chars in stdin. Return 0. */
int main(void)
{  int c;
   int charCount = 0;
   c = getchar();
   while (c != EOF)
   {  charCount++;
      c = getchar();
   }
   printf("%d\n", charCount);
   return 0;
}
```

- Removes comments
- Handles preprocessor directives
- Preprocessor replaces #include <stdio.h> with contents of /usr/include/stdio.h
- Preprocessor replaces EOF with -1

Step 1 - Preprocessing

`gcc -E charcount.c > charcount.i`

```
#include <stdio.h>
/* Write to stdout the number of
   chars in stdin. Return 0. */
int main(void)
{  int c;
   int charCount = 0;
   c = getchar();
   while (c != -1)
   {  charCount++;
      c = getchar();
   }
   printf("%d\n", charCount);
   return 0;
}
```

- Removes comments

After Preprocessing

charcount.i

```
...
int getchar();
int printf(char *fmt, ...);
...
int main(void)
{   int c;
    int charCount = 0;
    c = getchar();
    while (c != -1)
    {   charCount++;
        c = getchar();
    }
    printf("%d\n", charCount);
    return 0;
}
```

Why **int** instead of **char**?

- C language
- Missing comments
- Missing preprocessor directives
- Contains code from stdio.h
  - **Declarations** of getchar() and printf()
- Missing **definitions** of getchar() and printf()

Step 2 - Compiling

`gcc –S charcount.i`

```
...
int getchar();
int printf(char *fmt, ...);
...
int main(void)
{  int c;
   int charCount = 0;
   c = getchar();
   while (c != -1)
   {  charCount++;
      c = getchar();
   }
   printf("%d\n", charCount);
   return 0;
}
```

- Translate from C to assembly language
- Use function declarations to check calls of getchar() and printf()

- Compiler sees function declarations
- So compiler has enough information to check subsequent calls of getchar() and printf()

Step 2 - Compiling

`gcc –S charcount.i`

```
...
int getchar();
int printf(char *fmt, ...);
...
int main(void)
{   int c;
    int charCount = 0;
    c = getchar();
    while (c != -1)
    {   charCount++;
        c = getchar();
    }
    printf("%d\n", charCount);
    return 0;
}
```

- Definition of main() function
- Compiler checks calls of getchar() and printf() when encountered
- Compiler translates to assembly language

After Compiling

`charcount.s`

- Assembly language
- Missing definitions of getchar() and printf()

```
        .section ".rodata"
format:
        .string "%d\n"
        .section ".text"
        .globl main
        .type main,@function
main:
        pushq %rbp
        movq %rsp, %rbp
        subq $4, %rsp
        call getchar
loop:
        cmpl $-1, %eax
        je endloop
        incl -4(%rbp)
        call getchar
        jmp loop
endloop:
        movq $format, %rdi
        movl -4(%rbp), %esi
        movl $0, %eax
        call printf
        movl $0, %eax
        movq %rbp, %rsp
        popq %rbp
        ret
```

Step 3 - Assembling

```
gcc -c charcount.s
```

```
charcount.o
```

- Translate from assembly language to machine language…
- Machine language
- Missing definitions of getchar() and printf()

Step 4 - Linking

```
gcc -S charcount.o -o charcount
```

charcount

- Resolve references
- Fetch machine language code from the standard C library (/usr/lib/libc.a) to make the program complete

- Machine language contains definitions of getchar() and printf()
- Complete & Executable!

Interface

Character representation: ASCII

Mapping from integer values to characters on pretty much all machines:
ASCII (American Standard Code for Information Interchange) (/'æski/)

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	NUL									HT	LF					
16																
32	SP	!	"	#	$	%	&	'	(	)	*	+	,	-	.	/
48	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
64	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
80	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_
96	`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
112	p	q	r	s	t	u	v	w	x	y	z	{	|	}	~	

Notes: Many non-printing characters left blank in table above
UPPER-CASE and lower-case letters are 32 apart
… but they're internally contiguous. So are digits 0 through 9.

Character representation: EBCDIC

Extended Binary Coded Decimal Interchange Code (/ˈɛbsɪdɪk/)

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	NUL					HT										
16																
32						LF										
48																
64	SP											.	<	(	+	|
80	&										!	$	*	)	;	
96	-	/									|	,	%	_	>	?
112										`	:	#	@	'	=	"
128		a	b	c	d	e	f	g	h	i		{				
144		j	k	l	m	n	o	p	q	r		}				
160		~	s	t	u	v	w	x	y	z						
176																
192		A	B	C	D	E	F	G	H	I						
208		J	K	L	M	N	O	P	Q	R						
224	\		S	T	U	V	W	X	Y	Z						
240	0	1	2	3	4	5	6	7	8	9						

Partial map

Convert string to uppercase…

Are they good enough?

```
if ((c >= 97) && (c <= 122))
        c -= 32;
```

```
if ((c >= 'a') && (c <= 'z'))
        c += 'A' - 'a';
```

One Interface

- Provide a real **interface:**
  - Character data type
  - API for operations on characters
    - Works on all machines and representations
    - **Implemented** differently for different machines and representations (ASCII, EBCDIC, etc)

```
#include <ctype.h>

if (islower(c))
        c = toupper(c);
```

One Interface

```
#include <stdio.h>
#include <ctype.h>
int main(void)
{
    int c;
    while ((c = getchar()) != EOF) {
        if (islower(c))
            c = toupper(c);
        putchar(c);
    }
    return 0;
}
```

One Interface from ctype.h

```
$ man toupper
NAME
       toupper, tolower - convert letter to upper or lower case

SYNOPSIS
       #include <ctype.h>
       int toupper(int c);
       int tolower(int c);

DESCRIPTION
       toupper() converts the letter c to upper case, if possible.
       tolower() converts the letter c to lower case, if possible.

       If c is not an unsigned char value, or EOF, the behavior of
       these functions is undefined.

RETURN VALUE
       The value returned is that of the converted letter,
       or c if the conversion was not possible.
```